In [36]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVector
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
import re
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\anjal\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\anjal\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\anjal\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\anjal\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Out[36]: True

In [37]:
```
data = pd.read_csv('train.csv')
```

In [38]:
```
data.head()
```

Out[38]:

	qid	question_text	target
0	00002165364db923c7e6	How did Quebec nationalists see their province...	0
1	000032939017120e6e44	Do you have an adopted dog, how would you enco...	0
2	0000412ca6e4628ce2cf	Why does velocity affect time? Does velocity a...	0
3	000042bf85aa498cd78e	How did Otto von Guericke used the Magdeburg h...	0
4	0000455dfa3e01eae3af	Can I convert montra helicon D to a mountain b...	0

In [39]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1306122 entries, 0 to 1306121
Data columns (total 3 columns):
 #   Column         Non-Null Count    Dtype 
---  ------         --------------    ----- 
 0   qid            1306122 non-null  object
 1   question_text  1306122 non-null  object
 2   target         1306122 non-null  int64 
dtypes: int64(1), object(2)
memory usage: 29.9+ MB
```

In [40]:
```
# Cutting down data as data is too large to process
rows_to_delete = 1256122
# Randomly select rows to delete
rows_indices_to_delete = data.sample(n=rows_to_delete).index

# Drop the selected rows from the DataFrame
data = data.drop(rows_indices_to_delete)

# Reset the index after removing the rows
data = data.reset_index(drop=True)
```

In [41]:
```
data.describe()
```

Out[41]:

	target
count	50000.000000
mean	0.061400
std	0.240065
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

In [42]:
```
def tokenize_text(text):
    return word_tokenize(text)
```

In [43]:
```
def preprocess_token(token):
    token = token.lower()
    token = re.sub(r'[^a-zA-Z0-9]', '', token)
    return token
```

In [44]:
```
def remove_stopwords(tokens):
    stop_words = set(stopwords.words('english'))
    return [token for token in tokens if token not in stop_words]
```

In [45]:
```
def stem_tokens(tokens):
    stemmer = PorterStemmer()
    return [stemmer.stem(token) for token in tokens]
```

In [46]:
```
def lemmatize_tokens(tokens):
    lemmatizer = WordNetLemmatizer()
    return [lemmatizer.lemmatize(token) for token in tokens]
```

In [47]:
```
def preprocess_data(data):
    data['question_text'] = data['question_text'].apply(lambda x: x.lowe
    data['question_text'] = data['question_text'].apply(tokenize_text)  
    data['question_text'] = data['question_text'].apply(lambda tokens: [
    data['question_text'] = data['question_text'].apply(remove_stopwords
    data['question_text'] = data['question_text'].apply(stem_tokens)  # 
    data['question_text'] = data['question_text'].apply(lemmatize_tokens
    data['question_text'] = data['question_text'].apply(lambda x: ' '.jo
```

In [48]:
```
preprocess_data(data)
```

Data visualisation

In [19]:
```
import matplotlib.pyplot as plt
```

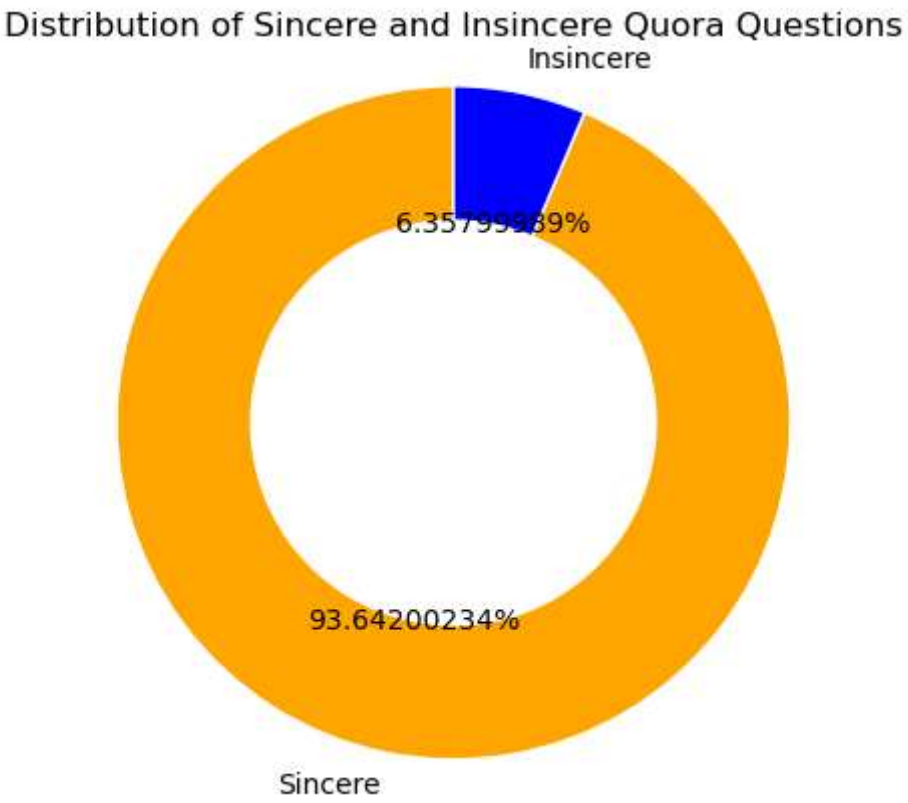
In [41]:
```
sincere_count=(data["target"]==0).sum()
print(sincere_count)
```

```
46821
```

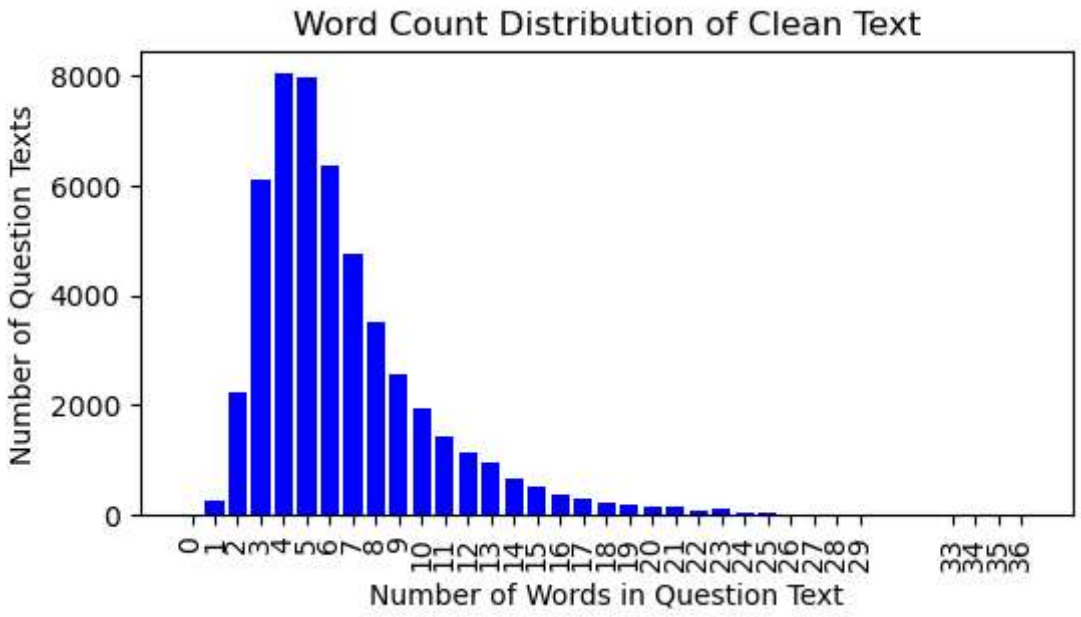
In [42]:
```
insincere_count=(data["target"]==1).sum()
print(insincere_count)
```

```
3179
```

In [43]:
```
# Create a list of the counts and labels
counts = [sincere_count, insincere_count]
labels = ['Sincere', 'Insincere']
colors = ['orange', 'blue']  # You can customize colors if needed

# Create a donut pie chart
plt.pie(counts, labels=labels, colors=colors, autopct='%1.8f%%', startang

# Draw a white circle in the middle to create the donut chart
centre_circle = plt.Circle((0, 0), 0.6, fc='white')
plt.gca().add_artist(centre_circle)

# Equal aspect ratio ensures that the pie chart is drawn as a circle
plt.axis('equal')

# Set chart title
plt.title('Distribution of Sincere and Insincere Quora Questions')

# Show the chart
plt.show()
```



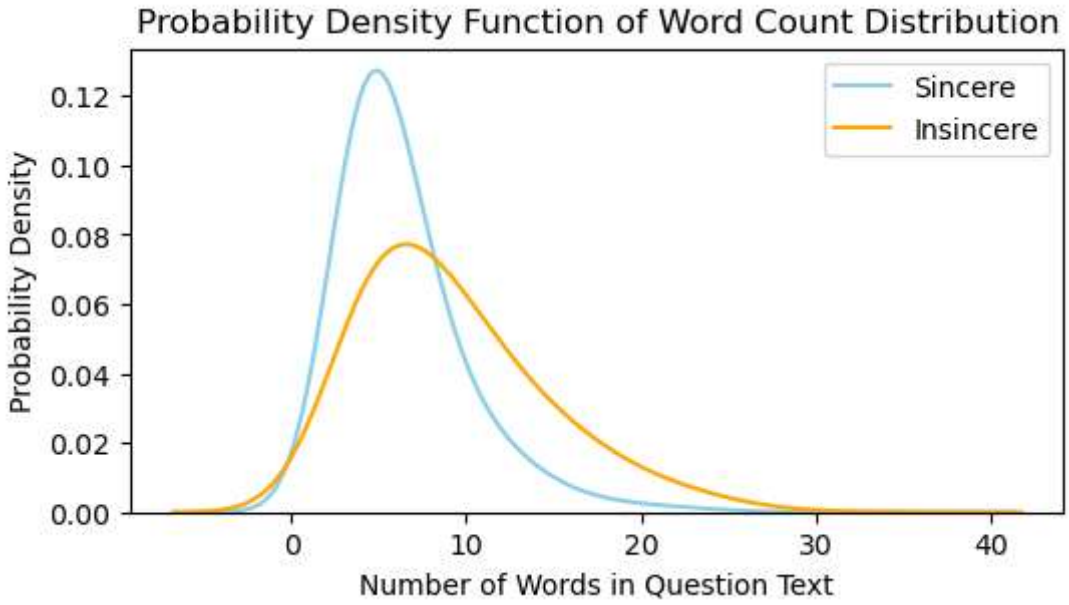
In [44]:
```
# Calculate the word count for each question text
data['word_count'] = data['question_text'].apply(lambda x: len(x.split()

# Calculate the frequency of word counts
word_count_frequency = data['word_count'].value_counts().reset_index()
word_count_frequency.columns = ['word_count', 'count']

# Sort by word count for better visualization
word_count_frequency = word_count_frequency.sort_values(by='word_count')

# Plot the bar graph with increasing bars towards the right
plt.figure(figsize=(6,3))
plt.bar(word_count_frequency['word_count'], word_count_frequency['count'
plt.xlabel('Number of Words in Question Text')
plt.ylabel('Number of Question Texts')
plt.title('Word Count Distribution of Clean Text')
plt.xticks(word_count_frequency['word_count'], rotation=90)
plt.show()
```



Bivariate Analysis

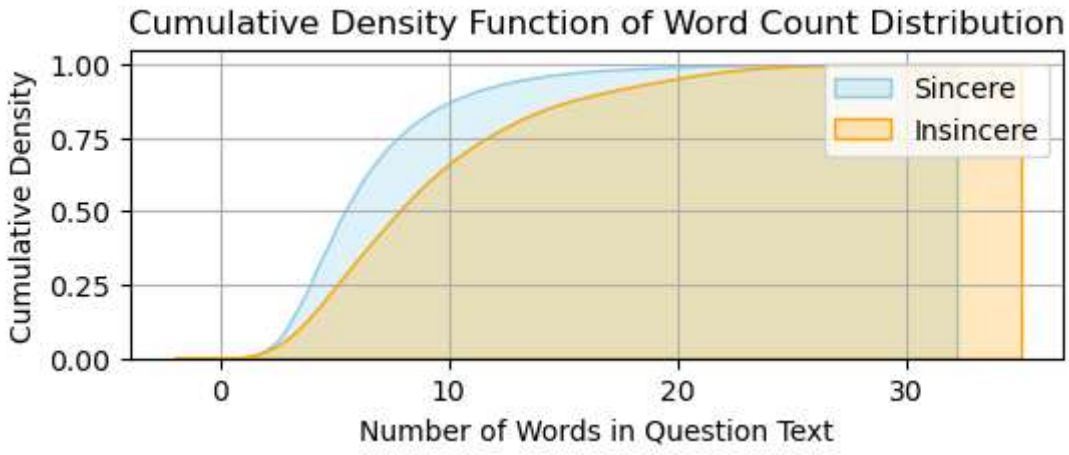
Probability density function: Clean Text Distribution

In [45]:
```
import seaborn as sns
# Separate sincere and insincere question texts
sincere_texts = data[data['target'] == 0]['question_text']
insincere_texts = data[data['target'] == 1]['question_text']

# Calculate the word count for each class
sincere_word_counts = sincere_texts.apply(lambda x: len(x.split()))
insincere_word_counts = insincere_texts.apply(lambda x: len(x.split()))

# Plot the probability density function (PDF) for both classes
plt.figure(figsize=(6,3))
sns.kdeplot(sincere_word_counts, color='skyblue', label='Sincere', bw_me
sns.kdeplot(insincere_word_counts, color='orange', label='Insincere', bw_
plt.xlabel('Number of Words in Question Text')
plt.ylabel('Probability Density')
plt.title('Probability Density Function of Word Count Distribution')
plt.legend()
plt.show()
```



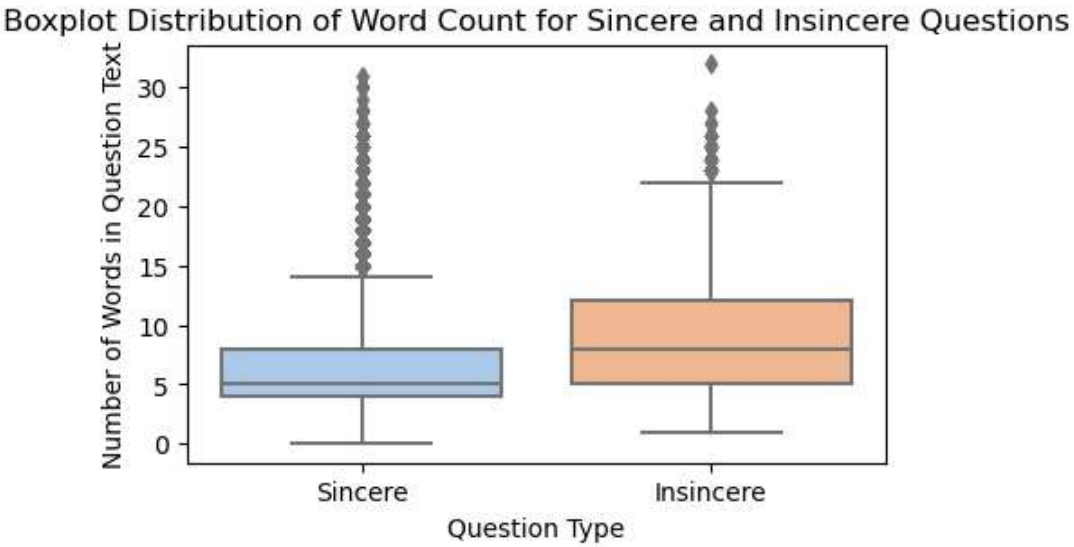
Cumulative Density Function

In [45]:
```
# Plot the cumulative density function (CDF) for both classes
plt.figure(figsize=(6, 2))
sns.kdeplot(sincere_word_counts, color='skyblue', label='Sincere', cumul
sns.kdeplot(insincere_word_counts, color='orange', label='Insincere', cu
plt.xlabel('Number of Words in Question Text')
plt.ylabel('Cumulative Density')
plt.title('Cumulative Density Function of Word Count Distribution')
plt.legend()
x_ticks = list(range(0, max(max(sincere_word_counts), max(insincere_word
plt.xticks(x_ticks)

# Add grid lines
plt.grid(True)

plt.show()
```



Boxplot Distribution

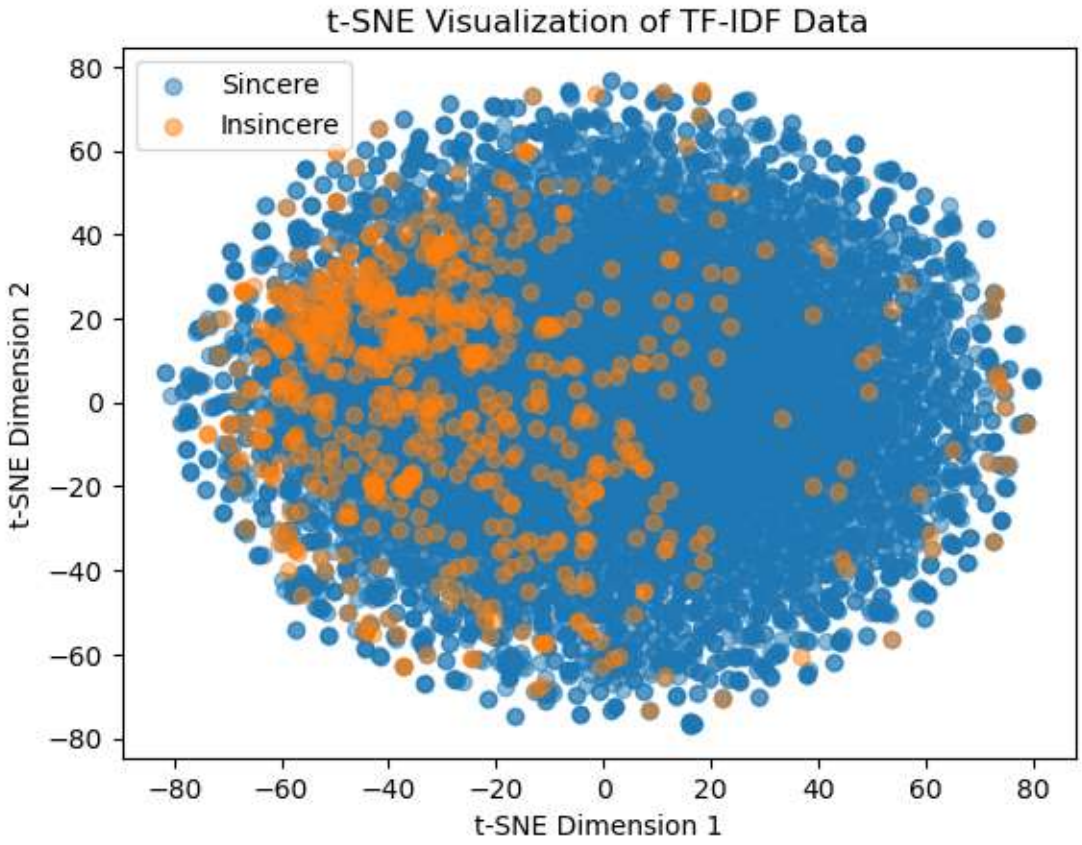
In [49]:
```
# Plot the boxplot distribution for both classes side by side
plt.figure(figsize=(5,3))
sns.boxplot(x='target', y='word_count', data=data, palette='pastel')
plt.xticks([0, 1], ['Sincere', 'Insincere'])
plt.xlabel('Question Type')
plt.ylabel('Number of Words in Question Text')
plt.title('Boxplot Distribution of Word Count for Sincere and Insincere Q
plt.show()
```



t-Distributed Stochastic Neighbour Embedding

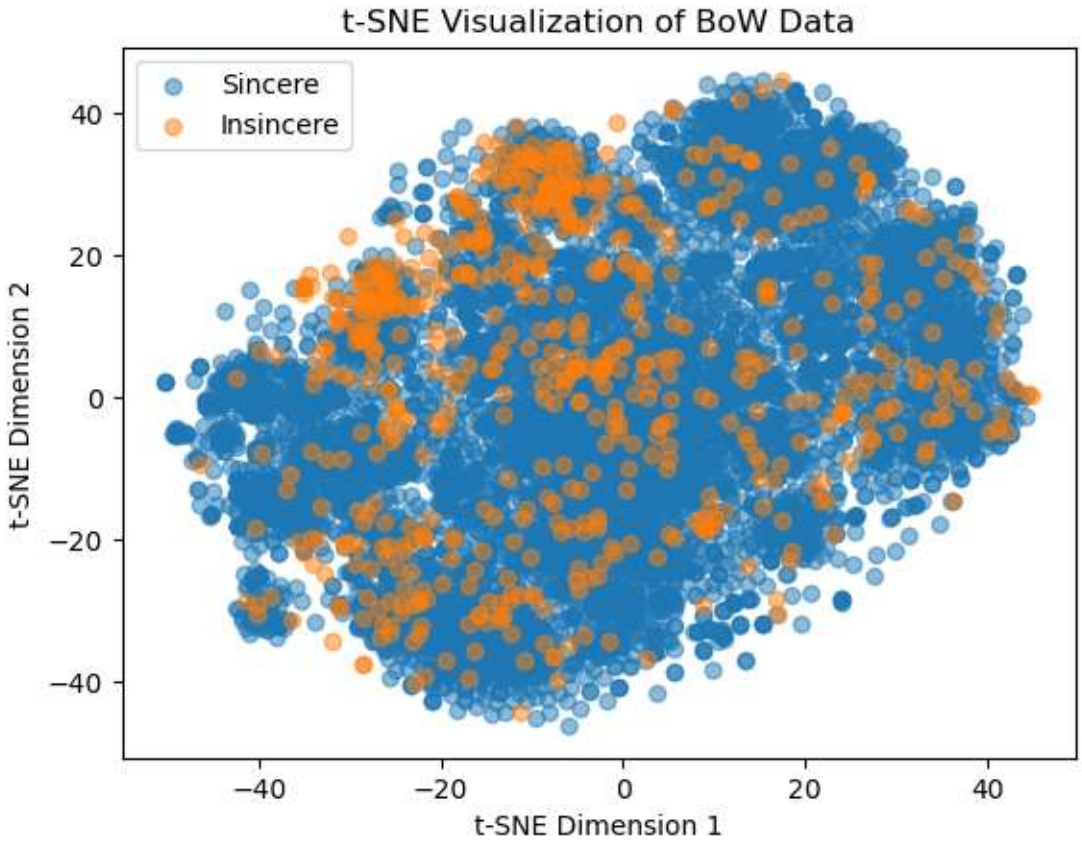
In [25]:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.manifold import TSNE

# Combine sincere and insincere question texts
all_texts = data['question_text']

# Sample a smaller subset of the data for faster execution
sample_size = 10000
sample_texts = all_texts.sample(sample_size, random_state=42)

# Create TF-IDF representation using TfidfVectorizer
vectorizer_tfidf = TfidfVectorizer()
X_tfidf = vectorizer_tfidf.fit_transform(sample_texts)

# Perform t-SNE on TF-IDF representation (using Barnes-Hut approximation,
tsne_tfidf = TSNE(n_components=2, random_state=42, method='barnes_hut')
X_tsne_tfidf = tsne_tfidf.fit_transform(X_tfidf.toarray())

# Separate t-SNE results for sincere and insincere questions
sincere_tsne_tfidf = X_tsne_tfidf[data['target'].sample(sample_size, ran
insincere_tsne_tfidf = X_tsne_tfidf[data['target'].sample(sample_size, r

# Plot t-SNE results
plt.scatter(sincere_tsne_tfidf[:, 0], sincere_tsne_tfidf[:, 1], label='S
plt.scatter(insincere_tsne_tfidf[:, 0], insincere_tsne_tfidf[:, 1], labe
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')
plt.title('t-SNE Visualization of TF-IDF Data')
plt.legend()
plt.show()
```


t-SNE Visualization of TF-IDF Data
Sincere
Insincere
t-SNE Dimension 2
t-SNE Dimension 1
80
60
40
20
0
−20
−40
−60
−80
−80
−60
−40
−20
0
20
40
60
80

In [26]:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
data = pd.read_csv('train.csv')

# Combine sincere and insincere question texts
all_texts = data['question_text']

# Sample a smaller subset of the data for faster execution
sample_size = 10000
sample_texts = all_texts.sample(sample_size, random_state=42)

# Create BoW representation using CountVectorizer
vectorizer_bow = CountVectorizer()
X_bow = vectorizer_bow.fit_transform(sample_texts)

# Perform t-SNE on BoW representation (using Barnes-Hut approximation)
tsne_bow = TSNE(n_components=2, random_state=42, method='barnes_hut')
X_tsne_bow = tsne_bow.fit_transform(X_bow.toarray())

# Separate t-SNE results for sincere and insincere questions
sincere_tsne_bow = X_tsne_bow[data['target'].sample(sample_size, random_
insincere_tsne_bow = X_tsne_bow[data['target'].sample(sample_size, rando

# Plot t-SNE results
plt.scatter(sincere_tsne_bow[:, 0], sincere_tsne_bow[:, 1], label='Since
plt.scatter(insincere_tsne_bow[:, 0], insincere_tsne_bow[:, 1], label='I
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')
plt.title('t-SNE Visualization of BoW Data')
plt.legend()
plt.show()
```



Building models over data

Logestic regression over TFIDF- Stemmed Text

In [5]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt
import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import wordnet
```

In [6]:
```
# Load your dataset (replace with your data loading code)
data = pd.read_csv('train.csv')
```

In [7]:
```
# Preprocessing function
def preprocess_text(text):
    # Tokenization
    tokens = word_tokenize(text)

    # Lowercasing and removing non-alphanumeric characters
    tokens = [re.sub(r'[^a-zA-Z0-9]', '', token.lower()) for token in to

    # Removing stopwords
    stop_words = set(stopwords.words('english'))
    tokens = [token for token in tokens if token not in stop_words]

    # Stemming
    stemmer = PorterStemmer()
    tokens = [stemmer.stem(token) for token in tokens]

    # Lemmatization
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(token, wordnet.VERB) for token in tok

    return ' '.join(tokens)
```

In [8]:
```
# Cutting down data as data is too large to process
rows_to_delete = 1256122
# Randomly select rows to delete
rows_indices_to_delete = data.sample(n=rows_to_delete).index

# Drop the selected rows from the DataFrame
data = data.drop(rows_indices_to_delete)

# Reset the index after removing the rows
data = data.reset_index(drop=True)
```

In [9]:
```
# Apply preprocessing to the text data
data['question_text'] = data['question_text'].apply(preprocess_text)
```

In [11]:
```
# Downsampling the majority class to balance the dataset
sincere_data = data[data['target'] == 0]
insincere_data = data[data['target'] == 1].sample(n=len(sincere_data), r
balanced_data = pd.concat([sincere_data, insincere_data])
```

In [12]:
```
# Split the balanced data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(balanced_data['quest
```

In [13]:
```
# Create TF-IDF representation using TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

In [44]:
```
# Hyperparameter tuning using GridSearchCV
param_grid = {'C': [0.001, 0.01, 0.1,0.5, 1,5]}  # List of values to sea
lr_model = LogisticRegression(max_iter=1000)  # Increase max_iter if nee
grid_search = GridSearchCV(lr_model, param_grid, cv=5, scoring='roc_auc'
grid_search.fit(X_train_tfidf, y_train)

# Get the best hyperparameters
best_c = grid_search.best_params_['C']
```

In [45]:
```
# Create the logistic regression model with the best hyperparameter
lr_model_tfidf = LogisticRegression(C=best_c, max_iter=1000)
```

In [46]:
```
# Train the model
lr_model_tfidf.fit(X_train_tfidf, y_train)
```

Out[46]:
```
LogisticRegression(C=5, max_iter=1000)
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

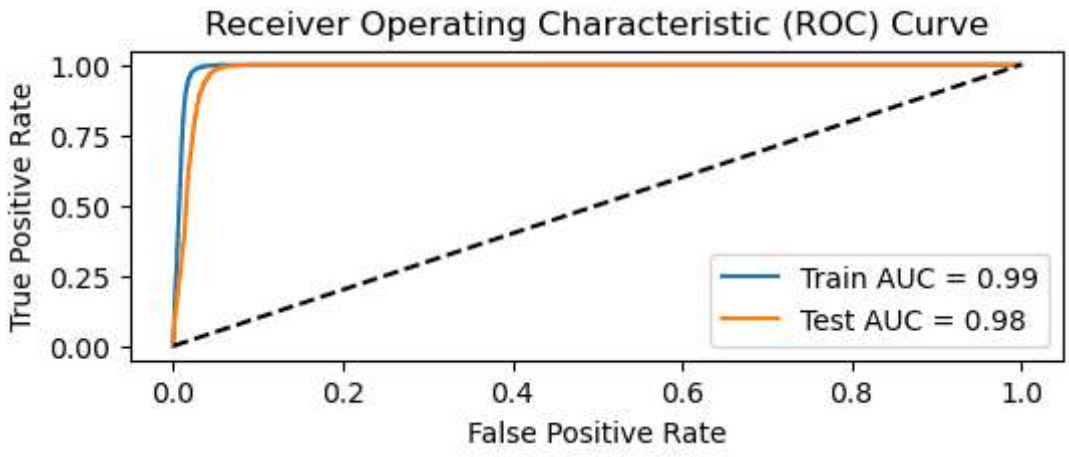
In [47]:
```
# Make predictions on train and test data
y_train_pred = lr_model_tfidf.predict_proba(X_train_tfidf)[:, 1]
y_test_pred = lr_model_tfidf.predict_proba(X_test_tfidf)[:, 1]
```

In [48]:
```
#Calculate AUC for train and test data
train_auc = roc_auc_score(y_train, y_train_pred)
test_auc = roc_auc_score(y_test, y_test_pred)
```

Model Evaluation

In [49]:

```
# Plot the ROC curve
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_pred)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_pred)

plt.figure(figsize=(6,2))
plt.plot(fpr_train, tpr_train, label=f'Train AUC = {train_auc:.2f}')
plt.plot(fpr_test, tpr_test, label=f'Test AUC = {test_auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.show()

# Print the best hyperparameter and its corresponding ROC AUC score
print("Best C:", best_c)
print("Best ROC AUC:", grid_search.best_score_)
```




```
Best C: 5
Best ROC AUC: 0.9849808064135923
```

Confusion Matrix and Classification Report

In [50]:

```
from sklearn.metrics import confusion_matrix, classification_report

# Set a threshold for predicting class labels from probabilities
threshold = 0.5
y_train_pred_binary = (y_train_pred >= threshold).astype(int)
y_test_pred_binary = (y_test_pred >= threshold).astype(int)

# Calculate the confusion matrix for train and test data
train_confusion_matrix = confusion_matrix(y_train, y_train_pred_binary)
test_confusion_matrix = confusion_matrix(y_test, y_test_pred_binary)

# Generate the classification report for train and test data
train_classification_report = classification_report(y_train, y_train_pre
test_classification_report = classification_report(y_test, y_test_pred_b

# Print the confusion matrix and classification report
print("Train Confusion Matrix:")
print(train_confusion_matrix)
print("\nTest Confusion Matrix:")
print(test_confusion_matrix)

print("\nTrain Classification Report:")
print(train_classification_report)
print("\nTest Classification Report:")
print(test_classification_report)
```

```
Train Confusion Matrix:
[[35866  1610]
 [  149 37386]]

Test Confusion Matrix:
[[8730  676]
 [  39 9308]]

Train Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.96      0.98     37476
           1       0.96      1.00      0.98     37535

    accuracy                           0.98     75011
   macro avg       0.98      0.98      0.98     75011
weighted avg       0.98      0.98      0.98     75011


Test Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.93      0.96      9406
           1       0.93      1.00      0.96      9347

    accuracy                           0.96     18753
   macro avg       0.96      0.96      0.96     18753
weighted avg       0.96      0.96      0.96     18753
```

Logestic regression over Bag of Words-Stemmed Text

In [51]:

```
from sklearn.feature_extraction.text import CountVectorizer

# Create Bag of Words (BoW) representation using CountVectorizer
bow_vectorizer = CountVectorizer()
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)
```

In [58]:
```
# Hyperparameter tuning using GridSearchCV
param_grid = {'C': [0.001, 0.01, 0.1,0.5]}  # List of values to search
lr_model = LogisticRegression(max_iter=1000)  # Increase max_iter if nee
grid_search = GridSearchCV(lr_model, param_grid, cv=5, scoring='roc_auc'
grid_search.fit(X_train_bow, y_train)
```

Out[58]:
```
GridSearchCV(cv=5, estimator=LogisticRegression(max_iter=1000),
             param_grid={'C': [0.001, 0.01, 0.1, 0.5]}, scoring='roc_au
c')
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [59]:
```
# Get the best hyperparameters
best_c = grid_search.best_params_['C']
```

In [60]:
```
# Create the logistic regression model with the best hyperparameter
lr_model_bow = LogisticRegression(C=best_c, max_iter=1000)
```

In [61]:
```
# Train the model
lr_model_bow.fit(X_train_bow, y_train)
```

Out[61]:
```
LogisticRegression(C=0.5, max_iter=1000)
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

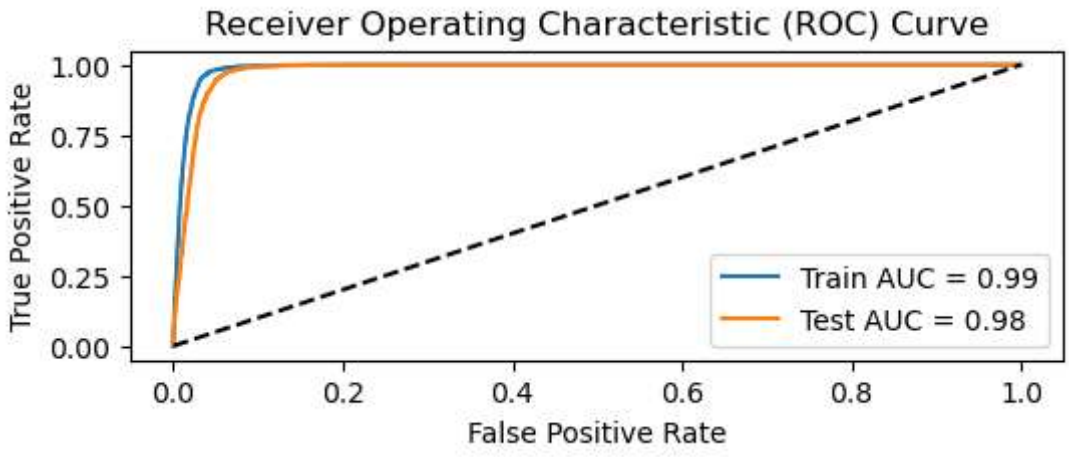
In [62]:
```
# Make predictions on train and test data
y_train_pred = lr_model_bow.predict_proba(X_train_bow)[:, 1]
y_test_pred = lr_model_bow.predict_proba(X_test_bow)[:, 1]
```

In [63]:
```
# Calculate AUC for train and test data
train_auc = roc_auc_score(y_train, y_train_pred)
test_auc = roc_auc_score(y_test, y_test_pred)
```

Model Evaluation

In [64]:

```
# Plot the ROC curve
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_pred)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_pred)

plt.figure(figsize=(6,2))
plt.plot(fpr_train, tpr_train, label=f'Train AUC = {train_auc:.2f}')
plt.plot(fpr_test, tpr_test, label=f'Test AUC = {test_auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.show()

# Print the best hyperparameter and its corresponding ROC AUC score
print("Best C:", best_c)
print("Best ROC AUC:", grid_search.best_score_)
```




```
Best C: 0.5
Best ROC AUC: 0.9794758701208416
```

Confusion Matrix and Classification Report

In [65]:

```
# Calculate confusion matrix and classification report
threshold = 0.5
y_train_pred_binary = (y_train_pred >= threshold).astype(int)
y_test_pred_binary = (y_test_pred >= threshold).astype(int)

train_confusion_matrix = confusion_matrix(y_train, y_train_pred_binary)
test_confusion_matrix = confusion_matrix(y_test, y_test_pred_binary)

train_classification_report = classification_report(y_train, y_train_pre
test_classification_report = classification_report(y_test, y_test_pred_b
```

In [40]:
```
# Print the confusion matrix and classification report
print("Train Confusion Matrix:")
print(train_confusion_matrix)
print("\nTest Confusion Matrix:")
print(test_confusion_matrix)

print("\nTrain Classification Report:")
print(train_classification_report)
print("\nTest Classification Report:")
print(test_classification_report)
```

```
Train Confusion Matrix:
[[35859  1617]
 [  471 37064]]

Test Confusion Matrix:
[[8762  644]
 [ 138 9209]]

Train Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.96      0.97     37476
           1       0.96      0.99      0.97     37535

    accuracy                           0.97     75011
   macro avg       0.97      0.97      0.97     75011
weighted avg       0.97      0.97      0.97     75011


Test Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.93      0.96      9406
           1       0.93      0.99      0.96      9347

    accuracy                           0.96     18753
   macro avg       0.96      0.96      0.96     18753
weighted avg       0.96      0.96      0.96     18753
```

XGBoost over Bag of Words- Stemmed Text

In [165]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import roc_auc_score

# Define the parameter grid
param_dist = {
    'learning_rate': [0.01, 0.1],
    'max_depth': [5, 7, 10,30],
    'n_estimators': [40,100, 200],
    'subsample': [0.8, 0.9, 1.0],
    'colsample_bytree': [0.8, 0.9, 1.0],
    'gamma': [0, 1, 2]
}
```

In [166]:
```
# Create XGBoost classifier
xgb_model = xgb.XGBClassifier(objective='binary:logistic')
```

In [167]:
```
# Perform randomized search with early stopping
random_search = RandomizedSearchCV(
    xgb_model,
    param_distributions=param_dist,
    n_iter=20,  # Number of random combinations to try
    cv=3,
    scoring='roc_auc',
    n_jobs=-1,  # Use all available CPU cores
    verbose=2,
    random_state=42
)
```

In [168]:
```
# Fit the model with early stopping
random_search.fit(X_train_bow, y_train, early_stopping_rounds=10, eval_m
```

```
Fitting 3 folds for each of 20 candidates, totalling 60 fits

C:\Users\anjal\AppData\Roaming\Python\Python310\site-packages\xgboo
st\sklearn.py:835: UserWarning: `eval_metric` in `fit` method is de
precated for better compatibility with scikit-learn, use `eval_metr
ic` in constructor or`set_params` instead.
  warnings.warn(
C:\Users\anjal\AppData\Roaming\Python\Python310\site-packages\xgboo
st\sklearn.py:835: UserWarning: `early_stopping_rounds` in `fit` me
thod is deprecated for better compatibility with scikit-learn, use
`early_stopping_rounds` in constructor or`set_params` instead.
  warnings.warn(

[0]     validation_0-auc:0.76218
[1]     validation_0-auc:0.80806
[2]     validation_0-auc:0.81005
[3]     validation_0-auc:0.81223
[4]     validation_0-auc:0.81506
[5]     validation_0-auc:0.82049
[6]     validation_0-auc:0.82281
```

AUC score

In [171]:
```
# Print the best hyperparameters and corresponding ROC AUC score
print("Best Hyperparameters:", random_search.best_params_)
print("Best ROC AUC:", random_search.best_score_)
```

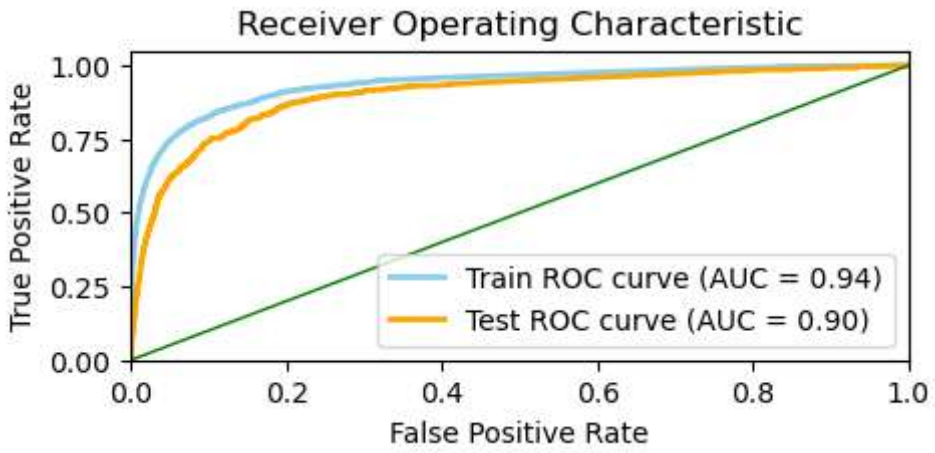
```
Best Hyperparameters: {'subsample': 0.8, 'n_estimators': 200, 'max_dept
h': 30, 'learning_rate': 0.1, 'gamma': 2, 'colsample_bytree': 0.9}
Best ROC AUC: 0.8980423019205528
```

In [174]:
```
# Get predicted probabilities for both train and test sets
y_train_pred_probs = random_search.predict_proba(X_train_bow)[:, 1]
y_test_pred_probs = random_search.predict_proba(X_test_bow)[:, 1]
```

In [175]:
```
# Calculate ROC curves and AUC values
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_pred_probs)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_pred_probs)
roc_auc_train = roc_auc_score(y_train, y_train_pred_probs)
roc_auc_test = roc_auc_score(y_test, y_test_pred_probs)
```

Model Evaluation

In [188]:
```
# Plot ROC curves
plt.figure(figsize=(5,2))
plt.plot(fpr_train, tpr_train, color='skyblue', lw=2, label='Train ROC c
plt.plot(fpr_test, tpr_test, color='orange', lw=2, label='Test ROC curve
plt.plot([0, 1], [0, 1], color='green', lw=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc='lower right')
plt.show()
```



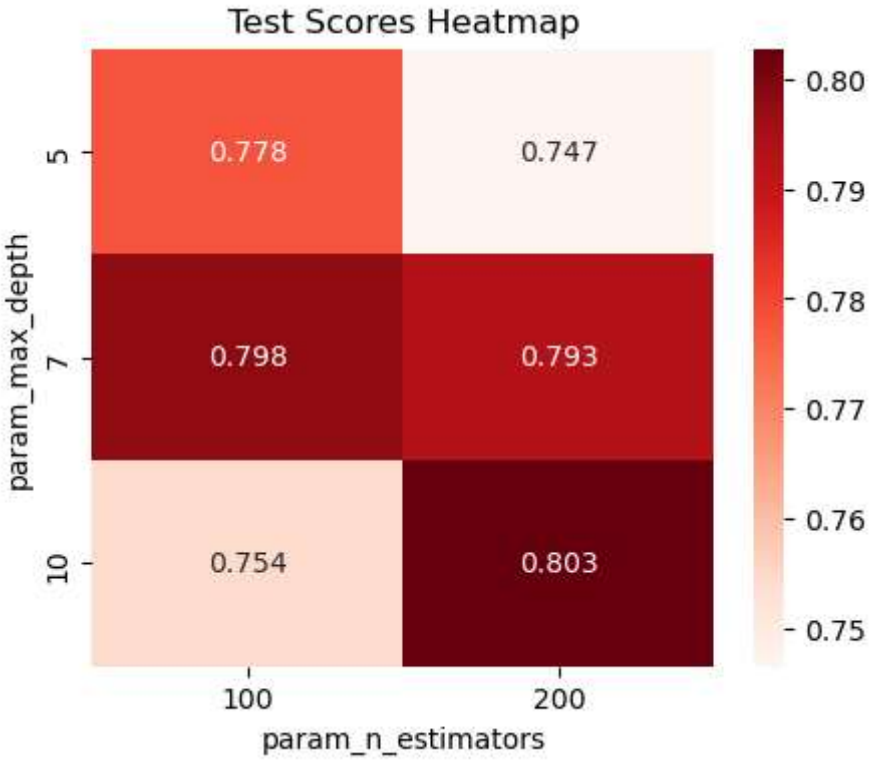
Heatmaps

In [95]:
```
# Extract the results from the randomized search
results = random_search.cv_results_
```

In [124]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create a DataFrame from the RandomizedSearchCV results
results_df = pd.DataFrame(random_search.cv_results_)
```

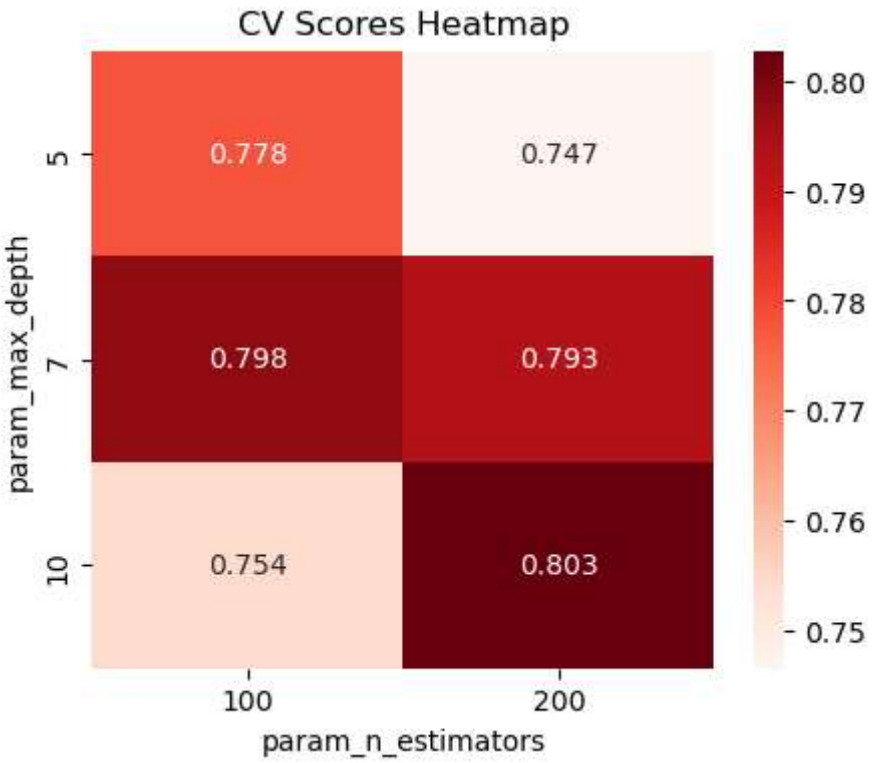
In [162]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming your DataFrame is named results_df
results_df['param_max_depth'] = results_df['param_max_depth'].astype(int

# Pivot the DataFrame for test scores with aggregation
test_pivot_df = results_df.pivot_table(index='param_max_depth', columns=

# Create a heatmap for test scores
plt.figure(figsize=(5,4))
sns.heatmap(test_pivot_df, annot=True, fmt=".3f", cmap="Reds")
plt.title('Test Scores Heatmap')
plt.xlabel('param_n_estimators')
plt.ylabel('param_max_depth')
plt.show()
```



In [161]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming your DataFrame is named results_df
results_df['param_max_depth'] = results_df['param_max_depth'].astype(int

# Pivot the DataFrame for CV scores with aggregation
cv_pivot_df = results_df.pivot_table(index='param_max_depth', columns='p

# Create a heatmap for CV scores with a red color palette
plt.figure(figsize=(5,4))
sns.heatmap(cv_pivot_df, annot=True, fmt=".3f", cmap="Reds")
plt.title('CV Scores Heatmap')
plt.xlabel('param_n_estimators')
plt.ylabel('param_max_depth')
plt.show()
```



Confusion Matrix

In [195]:
```
# Get predictions
y_train_pred = random_search.predict(X_train_bow)
y_test_pred = random_search.predict(X_test_bow)

# Calculate confusion matrices
conf_matrix_train = confusion_matrix(y_train, y_train_pred)
conf_matrix_test = confusion_matrix(y_test, y_test_pred)
print("Train Confusion Matrix:")
print(conf_matrix_train)

print("\nTest Confusion Matrix:")
print(conf_matrix_test)
```

```
Train Confusion Matrix:
[[37326   184]
 [ 1466  1024]]

Test Confusion Matrix:
[[9287   85]
 [ 471  157]]
```

Classification report

In [196]:
```
# Generate classification reports
class_report_train = classification_report(y_train, y_train_pred, target_
class_report_test = classification_report(y_test, y_test_pred, target_nar

print("Classification Report (Train):\n", class_report_train)
print("Classification Report (Test):\n", class_report_test)
```

```
Classification Report (Train):
               precision    recall  f1-score   support

    Negative       0.96      1.00      0.98     37510
    Positive       0.85      0.41      0.55      2490

    accuracy                           0.96     40000
   macro avg       0.90      0.70      0.77     40000
weighted avg       0.96      0.96      0.95     40000

Classification Report (Test):
               precision    recall  f1-score   support

    Negative       0.95      0.99      0.97      9372
    Positive       0.65      0.25      0.36       628

    accuracy                           0.94     10000
   macro avg       0.80      0.62      0.67     10000
weighted avg       0.93      0.94      0.93     10000
```

In [ ]:
```

```